Is inheritance bad practice in OOP?

Curtis Poe
Mostly I write Perl, JavaScript and SQL. Also assembler, C, COBOL, Java, Prolog.Updated 2y

Is inheritance bad practice in OOP? Many places that teach design patterns say to opt for composition over inheritance, but what about when multiple classes share logic from an abstract class such as in the Template Method design pattern?

Originally Answered: Is inheritance bad practice in OOP?
Short answer: yes, inheritance is usually a bad practice in OOP.

Longer answer: read on. The subject is complicated enough, and important enough, that there's really no way to give a brief answer and address these issues.

What follows is a very long read for Quora, but you've asked a question that people have argued about for fifty years, so giving you a simple answer would be unfair. Also, because I think this is a very important topic in our industry, I'm going to ask you for a favor. When you're done reading this, either click "Upvote" or leave an answer saying why you disagree. And please share this. Inheritance has its place, but it's so problematic that people need to be aware of these issues.

For a more complete answer, let's go back in time to 1967. That's when Simula 67 was released, offering the first "modern" object-oriented code. It's worth keeping in mind that this was almost 50 years ago (as of this writing) and one would *think* that we've learned a lot about OO programming, but in reality, we still have a lot more to learn.

In Simula 67 we had classes, polymorphism, encapsulation, and inheritance. For class-based OO, the first three have withstood the test of time and have been more or less "non-controversial" for five decades. Inheritance was difficult almost from day one. When you have a programming practice that people are arguing and debating over for fifty years, it's worth taking another look at it. In particular, given that Alan Kay, the inventor of the term "object-oriented programming", had serious reservations about inheritance, to the point where he left inheritance out of the definition altogether.

Got that? The computer scientist who named OO programming and is considered one of its inventors doesn't like inheritance. I think we'd be silly to dismiss his concerns.

Inheritance Violates Encapsulation

Here's just one example of why that's problematic. As a general rule, if a class is open for subclassing (I've noticed that declaring classes as "final" is mostly used as a performance hack and not for concrete business reasons), the class should not know about its descendants, though the descendants, of course, need to know about their ancestors. Now imagine that you have an inheritance chain where `Child` inherits from `Parent` inheriting from `GrandParent`. `Child` naturally relies on some behavior from `GrandParent`, but in reality, it should only be *aware* of the `Parent` class. This is because the contract presented by `Parent` should be

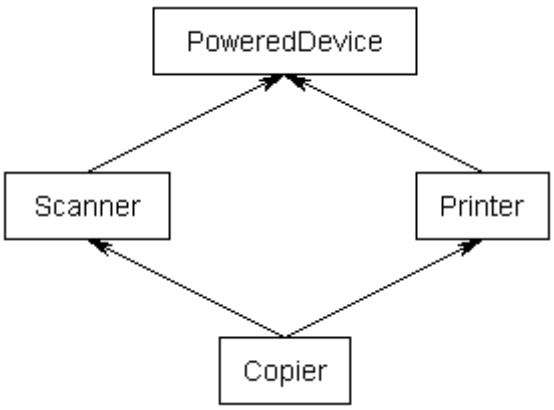
inviolate (open for extension, not change) and if `Parent` later decided to inherit from a different class, `Child` should still work. In practice, the more you inherit, the more behavior you're pulling in which you may not need and it's hard to be sure exactly where that behavior is coming from. This makes deep inheritance chains harder to debug and maintain.
So the author of the `Parent` class implements a public `foo()` method after verifying that the `GrandParent` class does not. The `Child` class now inherits `Parent.foo()`, but later the author of the `GrandParent` class says "I need to implement `GrandParent.foo()`". Since a class should generally not be aware of its descendants, they have no idea that this public method will be later overridden. The Child class, meanwhile, realizes they need `GrandParent.foo()` and not `Parent.foo()`, but whether or not they're able to call that method directly depends on the language. If
the `Child` class *can* call `GrandParent.foo()` directly, that's a gross encapsulation violation (they should only know about `Parent` and `Parent` should be free to inherit from another class). If they can't call it directly, they may be forced to cut-n-paste the behavior of `GrandParent.foo()`, or find some other awkward workaround.
And then `GrandChild` inherits from `Child` and wants to implement call `Parent.foo()`. Oops.
Because you've tightly coupled child classes to parent classes, you've violated encapsulation and the above is merely one example of how things can go wrong. You can hit Google for many other explanations of why inheritance violates encapsulation.

I might add that people are often in two camps about this. Those who write and release a lot of small modules/libraries/packages/whatever, often don't experience the above pain. However, those who work on very large scale systems (I do), are more likely to see these problems firsthand. It's like a beginner writing a 100 line script and not realizing the problems with global variables: certain bad practices don't scale, but until you scale, it can be hard to see why they're bad.

Multiple Inheritance is an Abomination

Single inheritance has many issues and I've hardly scratched the surface, but multiple inheritance (MI) is so problematic that many languages ban it outright, but try to offer some alternatives to it (e.g., Java and interfaces). Other languages, such as Perl, allow multiple inheritance, but you're generally warned not to use it. Why not?

Imagine the following inheritance hierarchy:



If your `Copier` paper jams, you might want to tell it to `halt()` to prevent damage. Fortunately, you don't have to write that method because it's already implemented

in `Printer`. Unfortunately, because inheritance usually is implemented via depth-first search of the inheritance tree, you've called the `PoweredDevice.halt()` method, removing power to the `Copier` altogether, losing all of the other queued jobs. Awesome!
There *is* something called C3 linearization which attempts to implement an intelligent breadth-first search of the inheritance hierarchy for the most suitable method for your needs, but all it does is mitigate the problem. It will find the next method that seems to most clearly match what your code is looking for; it can't guarantee that it will find the code that actually does what you need it to do. Thus, while it's a well-defined algorithm, it's still just a heuristic and heuristics break sooner or later. See my previous comments about "scale."

I might add that some find the term "C3 linearization" to be a little bit too "comp-sci speak" for them, so I offer an alternative name: "mopping the Titanic." When you have to jump through such extreme hoops to work around a problem, maybe it's better to fix the problem instead?

What is Inheritance?

A subclass is generally considered to be a more specific version of a parent class. For example, a *Dog* might be a subclass of *Mammal* which might, in turn, be a subclass of *Animal*. Dogs don't inherit from Trees just because you want a `bark()` method. Keep that in mind.
While I can't find the quote right now (apologies), the creators of the Beta programming language only supported single inheritance. When they were researching how to support multiple inheritance (MI) , they discovered that none of the examples they could find were about creating more specific versions of things. Instead, they were about sharing behaviors. So MI in the real world seemed to violate the intent of inheritance in general and the Beta developers just left it out (to be fair, there were practical considerations, too).

What does this mean? Well, the duckbilled platypus is a mammal, but it lays eggs. Animals that lay eggs are called "oviparous". Most mammals gestate their young in the womb and are "viviparous." Clearly it's true that a duckbilled platypus *isa* mammal, but it's not viviparous. If you wanted to model its reproductive cycle, how do you share that complex behavior without copying it? Are you going to use multiple inheritance? Maybe you have an `EggLaying` class (spiders use it too!). So you have:

```
1. class DuckBilledPlatypus isa Mammal isa EggLaying {
2.     ...
3. }
```

Except … `EggLaying` is an adjective, not a noun. Why is that even a class? And should you inherit from that before or after the other class?
While the example above is silly, it shows a critical problem with inheritance: real-world problems don't neatly fit the tree-like structure of inheritance.

But I hear you say "ah ha! That's a straw man because you forgot delegation which can easily solve this problem!"

No, I didn't forget delegation.

The Problem with Delegation

I like delegation and I use it, but it's also abused. Here's a real-world example.

A long time ago I worked for a company that needed to audit their objects to see who was changing the data. That was simple: they just had their objects inherit from their `Audited` class. One day they had an issue where they had a class which needed to be `Audited`, but only to write a line to a log file. It was inheriting from a class which did not have `Audited` in the inheritance hierarchy. They were forced to fall back on to multiple inheritance to implement this feature and it meant that the class in question also picked up some "extra" behaviors from `Audited` which it didn't need and definitely didn't want.
Eventually it was pointed out that Audited should be something we should delegate to instead of inheriting from.
This solution was better, but had the problem that:

1. You had to write extra scaffolding code to implement the delegation
2. And why is a pure behavior, sans state, a class in the first place?

Think about that last point. It makes sense to have a particular instance of, say, a house, or an order, or a dog, but what is an instance of an "audited"? That question is completely non-sensical *except* in the tortured terms of OO modeling that we have today.

What's the Root of the Problem?

Remember that we've been arguing about inheritance for *fifty years*. For anyone who disagrees with my points above, you can spend the next fifty years arguing with people who do agree with those points. Remember: *fifty years of arguing*. Even the most intransigent developer should pause at that.

The core of the problem with inheritance is that classes actually serve two purposes. First, they're agents of responsibility. Classes should be responsible for the things they're responsible for and as anyone working on large scale systems can tell you, classes take on more responsibility over time and this tends to *increase* class sizes.

Got that? Classes as agents of responsibility leads to *larger class sizes*.

However, when you inherit from a class, it's an agent of code reuse. If my `Dog` inherits from `Mammal`, I don't want to find out that someone's implemented `Mammal.play_mp3(filename)`. In fact, even if all of the `Mammal` methods are appropriate for that class, if I just want a `GuardDog`, there's a lot of behavior in the `Mammal` class I probably don't need or want for modeling.
Classes as agents of code reuse should prefer *smaller class sizes*.

So there's a fundamental tension in classes when used with inheritance. The secret is to decouple class responsibility and behavior. This sounds odd, but it's straightforward.

One way to "solve" this was interfaces, but that didn't quite work because interfaces merely declared responsibility without behavior and many developers have moaned about duplicated code in Java apps due to heavy use of interfaces (Java is now allowing interfaces to also provide an implementation, but this is only a partial fix).

Delegation was interesting, but it often leads to plenty of classes you can instantiate which, nonetheless, should not be classes (see the `Audited` example above).
Mixins, which came from a Lisp variant named "Flavors", were an interesting idea. They provide small bits of behavior you can "mix in" with any class you like. However, they don't

really offer any compositional safety and have the same encapsulation issues as inheritance, and similar issues to diamond inheritance.

Using Ruby as a more familiar example, if your `Person` class mixes in both your `Serializable` and `Storable` mixins and both of those have a `.marshal()` method, which one do you get? If it was multiple inheritance, you'd get the method from the first class you inherited from. With mixins, you get it from the *last* class you mixed in because Ruby implements mixins under the hood as single inheritance!

```
irb(main):026:0> Person.ancestors
=> [Person, Serializable, Storable, Object, Kernel]
```

Surprise! You can also watch this part of a talk I gave about OO programming where I describe some more issues with Ruby mixins.

Aspect-Oriented Programming (AOP) has been another attempt to solve this and while it was novel, it had a core problem: instead of action at a distance being considered a bug, AOP made it a first-class concept! I still remember a friend debugging a threaded AspectJ application which randomly failed. Seems some developer renamed a method such that a pointcut no longer matched and as a result, the advice could not be applied to that method, meaning it wasn't locked and different threads could access it at the same time.

If none of the AOP stuff that I wrote made any sense, let me paraphrase: AOP was an experiment that accidentally made its way into the wild and when found, should be beaten to death viciously with hammers. If you must use AOP, only apply advice to augment a program, not modify it. If the program won't run correctly without Aspects, that should be considered a bug — that's one of the reasons why aspects are often shown only used for logging or other trivial issues. (Side note: one of the original authors of the first AOP paper is a friend of mine. If you are he and you're reading this, we'll argue about it later).

The Solution

I strongly recommend Smalltalk-style traits. They were first described in Traits: Composable Units of Behavior and it's the paper that most seemed to rely on when they implemented traits in other languages. Unfortunately, they should be relying on Traits: The Formal Model, a paper which clarifies many things (but not all) which are vague in the first paper.

In short, traits–or "roles" as they're sometimes known–are composable units of behavior. They're like mixins, but the methods are added directly to your class and if another method of the same type (name + signature, but depends on the language) is found, the code fails to compile unless you *explicitly* state which implementation you need. No more heuristic guesswork.

So the class states what responsibilities it has, but any behavior which must be shared (such as serialization) goes into a trait/role.

There's a hell of a lot more than that, but I've routinely built large systems using only roles and no (or limited) inheritance and found the systems are more predictable, more flexible, and easier to understand. Further, you'll find more and more languages are implementing them.

For a shorter description, I wrote a Five Minute Introduction to Smalltalk-style Traits. It's rather limited, but it gives you the gist of the issue.

Update

Because I've been asked about this more than once: yes, I support using abstract base classes. In fact, that's just about the only time I support inheritance. However, even then I try to avoid them if I can use a trait instead. Traits gain me the compositional safety that inheritance cannot.

Ryan Mattes
· August 30, 2016
My only disagreement is that the examples of problems with inheritance are always actually poor design decisions and have nothing to do with inheritance actually being bad. Do you say screwdrivers are "bad" because they make poor hammers? Good design and standards avoid these problems every time.

Of course having spent enough time in the real world I know that bad design decisions are often forced, due to budget and timeline constraints, or are the result of inexperience and/or bad management. But I don't think that's a good reason to eliminate a tool, just because some people insist on using it for the wrong things.

Inheritance is insanely useful in some cases. You just have to use the right tool for the job.

Curtis Poe
· August 30, 2016
Ryan: I was considering that point when writing this post and the "Audited" example is a perfect example of a poor design decision. Given that we have 50 years of awful design decisions, it's not clear to me that we can merely blame the software developers; that's too easy a thing to do. Instead, I would suggest that OO has managed to make it too easy to create poor designs without offering natural guidance on how to create good ones.

Also, I don't advocate eliminating inheritance. I use it from time to time when it's the right tool, but more often than not I find myself wanting to share behavior (roles) and not want a more specialized version of a thing (subclass).
Ryan Mattes
I would agree that we need another thing, standardized across languages, like interfaces that carry implementation, to make it easier to do it right. I'm just tired of the "inheritance sucks" band wagon, because everyone's reason is always that they've done it wrong.

Thomas Thrien
· September 28, 2018
"Inheritance is insanely useful in some cases. You just have to use the right tool for the job."

I think that's the key phrase: as said already by Curtis Poe in his main answer, inheritance may (and will, to my own experience) work fine for small projects and/or closed libraries, but on large scale projects where any layer of the inheritance tree (I could not find a better wording) can be changed by a team that has no clue (and is not meant to have any clue) about the layers above (farer away from the root), inheritance is calling for trouble.

Of course you can work around it, but again, "workaround" is the key phrase *here*: usually a *workaround* is not a real *solution*.

Against knowing better, I chose inheritance too often, believing that the project is limited enough to be handled with it, and had to learn afterwards, that things are always greater than they appear.

But nevertheless, I agree in the statement, it is a tool that still needs to be considered, although it should be kept in mind that this tool has sharped edges.

Jeff Gib

Well, deep hierarchies are bad design. And I would never think it acceptable for a developer not to have access to the entire inheritance hierarchy. If I were to see even three layers in a hierarchy, better be a really good reason.

Anton Carver

· September 26, 2017

I am pretty sure that in a few years we will be hearing why traits are bad and evil. The evidence will of course be all kinds of examples of there misuse by misguided programmers (who thought they were the new silver bullet) and of course they will ignore all the cases where they were used well to solve real problems. Why do people not understand that bad programmers will write bad code in any language, paradigm, method and stop blaming the tools.

Curtis Poe

· September 26, 2017

Traits are certainly not without issue, but we've been using them for well over a decade in the Perl community and unlike classes, which are often hard to pin down, traits are much easier to describe, conceptually. Thus, the systems I've seen which use them are generally far more coherent than traditional class-based systems.

You can certainly say what you want about traits, but it would be helpful to come up with a concrete example, rather than just going with your gut. I have real, first-hand experience using this technology for many years and while I'm generally skeptical of new technologies, this one is solid.

Anton Carver

I don't think you read what I said. I didn't say there was a problem with traits (just as there isn't with inheritance). It is what people do with it which is always the problem.

Andrew Rout

· August 29, 2016

Why is it that posting a cat photo and tired joke will get 10k upvotes, but great answers like this will inevitably fade away…well written, well thought out, and as you said, you barely even scratched the surface of it all. You could literally write a book on this topic (hint, hint)

Upvote

Curtis Poe

· August 29, 2016

I've already written two well-reviewed books (Beginning Perl: Curtis Poe: 9781118013847: Amazon.com: Books and coauthored Perl Hacks: Tips & Tools for Programming, Debugging, and Surviving 1, Chromatic, Damian Conway, Curtis "Ovid" Poe, Curtis (Ovid) Poe, eBook - Amazon.com), but they're terribly time-consuming and publishers are loathe to pick them up unless they have a *very* wide audience. It's frustrating.

Also, due to many years of experience blogging, I've discovered that there's a sweet spot of about two paragraphs which gets the best response. Only a couple of sentences and it's not taken seriously. Cover a situation full and in depth? People's eyes glaze over and they move on to something else. It's a serious problem when my "long" answer is considered a long answer :/

Jason Roos
· April 26, 2019
The question asked you about inheritance in a specific context: Template Method.

You then proceeded to lament the dangers of deep inheritance hierarchies and of multiple inheritance, neither of which are relevant to Template Method, which defines only a simple two-level, single inheritance relationship—if one even chooses to use inheritance to achieve it. (Template Method has no such requirement.)

Indeed, you did not even mention Template Method.

Curtis Poe
· April 26, 2019
The question is "Is inheritance bad practice in OOP?" All the words after that are expanding on the question. The reference to the Template Method is in the phrase "such as in the Template Method…". The words "such as" mean this is exemplifying phrase, not a qualifying one. Thus, I chose to deal with the fundamental question rather than the example.

That being said, yes, the "but what about" part is refining the question, but again, I chose to deal with the root question because once you understand the fundamental problems with inheritance, you can start to infer the rest. Specifically, if the base class follows the single responsibility principle, it might be OK to use it (this is helpful when dealing with sequencing issues that traits don't handle well), but otherwise, bear in mind that classes are for specialization and traits are for providing well-defined behavior. They are different tools with different uses.
Jason Roos
OK, but you responded immediately, "Hell, yes," as if to imply that inheritance is universally bad, but then you discussed only extreme abuses that many advocates of inheritance would find equally objectionable.

Chris Fox
· August 31, 2016
Multiple inheritance is an absolute trap and nobody should ever use it. Dropping it is one of the few things I liked about C#, but C# uses throw idiomatically so I don't write in it.

Shallow singe inheritance wasn't all bad but inheritance in general is one of those ideas that sounded good in technical specs but didn't work out so well in real life. Operator overloading sounded like a good idea but in the hands of morons it became a nightmare. Try and catch were good ideas too but then someone had to let developers throw as well when throw should have been a privileged machine instruction, not something a junior developer with no wisdom should be using throughout his code. At least with GOTO you knew where you were going to end up.

I'm back to plain old C these days. There are times that single inheritance one level deep would be nice but I can manage without it.

Erik Alapää
· February 26, 2019

Simple is almost always good. But if you know what you are doing, and think through your design, both implementation inheritance and multiple inheritance can be extremely useful in some cases. If you oversimplify, you get barren languages with too few abstractions and tools.

Matt Waters

This is a year later, but exceptions are fine for truly exceptional conditions. But if they're part of normal program flow, then they're a less predictable form of GOTO. Exceptions can actually be used as ON ERROR GOTO, which I've seen in BASIC code as well and is even worse than GOTO's. Throwing is always fine if you have an unexpected state. The try/catch/finally part is what gets you in trouble.

Nabarun Mondal

· December 9, 2017

Thank you.Thank you.Thank you.

Christopher F Clark

· Sun

I am still reading the traits formal model paper to see where I agree or disagree with you. So, I am for now doing both, upvoting and leaving a comment.

I am a big fan of inheritance for sharing implementation. To me, it is the ultimate form of the DRY (don't repeat yourself) rule. One should only implement each function once. And, it is possible that traits/roles mostly solve that, although I would want to be able to have traits that share code (and thus use some form of inheritance to achieve that, so again there is only one copy of the code).

I think I want something like sets of traits, where each function/member has one instance (you only get one definition of each name), but where you can add or subtract names to meet your requirements, but you don't have to do that with too much repetition.

And, to me the problem actually goes to the inability to properly see what the resulting code actually is, that it is more a visibility problem. That, when you have composed your object, you need to know what the resulting implementation actually is and what code is there. Much like people used to "expand macros" in C, but more subtly, e.g. how IDEs give you lists of functions you can call, so that you pick the one you want.

And, equally important at the final level to control that. I have more than once, seen that the issue is down in some base code, where it implements some pattern I want to replicate, but where there is some subtle thing where it either does the wrong thing, or fails to do the right thing. And, to me that was what AOP was intended to solve. The problem is one wants to do that in a non-spooky way, i.e. where the action at a distance is obvious at the point of use, not done via "magic" (at least not opaque magic).

Stephen Doherty

· March 28, 2019

I'm going to upvote and leave a comment just because this is such a well thought out article with links to other fantastic sources. Thank you for sharing this!

Aaron Mefford

· December 11, 2016

As I read your lead in I thought I would certainly disagree with your thesis, but would be worth the read. However as I read and you reminded me of many hours of very painful debugging, I found your comments compelling. I look forward to greater adoption of traits.

Richard Conto
· February 4, 2019
The first problem of (simple) inheritance you describe is more a matter of version control. By adding a new method/member after it was established, GrandParent changed the API. It's like adding a new element to an existing public structure or a new parameter to an existing function.

I agree with you about multiple inheritance.

As for the rest of your answer, I thank you for giving me something to consider - but for now I'm going to categorize it under issues of software and data structure design.

My first exposure to object oriented design was when I was exploring device drivers and filesystems - in assembler.

Prateek Chopra
· February 14, 2017
While I agree, that Single Inheritance is inherently complex and requires that people using it are skilled in OO concepts. However, they do offer a level of code re-usability and can address the problem of one variant change. I would as a rule not have anything over one level deep inheritance and would switch to composition if that's the case.

Also, I found certain loop holes in your arguments against Single Inheritance. Here are my reservation for them.

"**So the author of the Parent class implements a public foo() method after verifying that the GrandParent class does not. The Child class now inherits Parent.foo(), but later the author of the GrandParent class says "I need to implement GrandParent.foo()**"—You wrote

foo() should have been implemented by that concrete class, where it suited its abstraction. Thus, Parent shouldn't have implemented foo() in the first place if it made sense for GrandParent to implement. -- **Mistake 1.** In other words, say we have Rectangle implement Shape. Then method Area (foo()), shouldn't have been defined in Rectangle.

"**The Child class, meanwhile, realizes they need GrandParent.foo() and not Parent.foo(), but whether or not they're able to call that method directly depends on the language.**"

Mistake 1 above leading to another mistake. Ideally it shouldn't matter what the implementation details of foo() are as long as its meeting the contract.

Kirk Pepperdine
· October 3, 2017
Sorry but to absolutely say that inheritance is bad is simply wrong. I don't want to pick at this point by point but I'll take on one point, Inheritance is *not* a violation of encapsulation, it is an extension of existing behavior. Your Parent example is too abstract to comment about but it is horribly flawed in that if you inherit or encapsulate you'll still have to write a foo(). That

is unless the parent foo() does what you want and you can inherit that behavior instead of having to code a boilerplate pass through method.

Inheritance is a technique just as composition is a technique. Though I tend to favor composition, I have plenty of places where I do use inheritance.

Finally, I bet you violate encapsulation all the time in your code. I say this because all developers violate encapsulation at one point or another.

Kirk Augustin
· September 18, 2018
While it is true inheritance can be abused and over used, and that multiple inheritance is a bad idea, you can't do polymorphism without inheritance, so it is essential.

Greg Krsak
· August 29, 2016
This guy knows his stuff. But if you're doing Test-Driven Development (and you should be), then inheritance is never a problem.

Chris Fox
· August 31, 2016
TDD is a ridiculous fad. I'd end an interview if the manager mentioned it, an if he mentiioned pair programming I'd end it rudely. Anyone with experience in solving problems can see what it so ass-backward about TDD without ever doing it. You'll spend more time revisiting your silly tests than you'll spend writing the actual code.

Vance Feld
· August 30, 2016
What would unit tests solve in terms of design and multiple-inheritance? Your tests are still scaffolded on the same problem… that behavior is actually different if a descendant overrides the original implementation, meaning there is potentially undefined behavior (in terms of the test scenarios). Unless I'm missing something.

Akin Williams
· March 14, 2019
I never understood reason behind the classification of fatal errors in trait collisions until you just explained it. Nice answer.

Rob Anderson
· November 14, 2019
As an individual programmer, I rely on code reuse a lot. I have a base class, a way of doing link lists, and all my classes come from one base class. I agree that a depth of more than 2 needs to be justified. I have developed class generators, program generators and associated tools. Inheritance in my view is very important for persistence and automatic generation of classes and programs. Automatic generation of classes and programs is my preferred way to minimize debugging effort.

But as an individual programmer I am mostly concerned with my own understanding of the code. I know that nobody but me would like to work with these programs because they would have to deal with a lot of program fluff unrelated to the problem at hand.

In my experience managing a team doing C++ development I found that a logical class hierarchy made the code base understandable. OO design is much easier to understand than procedural code with data structures. If the problem suits the solution using inheritance then the code is made more understandable with inheritance, I have seen good examples with inheritance a few levels deep. The common characteristic is the addition of simple features to descendants who also need all the features of the ancestor.

Pablo De La Sotta
· September 13, 2016
Can I implement something similar to a smalltalk trait in a language that does not natively support it? Like Java or C#?

Curtis Poe
· September 14, 2016
More and more languages are getting support for traits: Trait (computer programming)

Jeffrey Efrain Ferreiras
· August 29, 2016
Very informative. Thank you for the thought out response. 50 Years! wow.

Alexis Rodriguez
· July 15, 2019
Isn't AOP sort of like decorators, LISP macros, and metaclasses? Function decorators can only modify the outside of a function call not the inside, although class decorators can add and change methods and fields. Metaclasses are like class decorators, but they can travel in the inheritance chain for extreme power, but I think they are sometimes needed and are less magical than eval. Macros (LISP) have extreme power allowing you to reuse code between many "languages" with semantics specialized to their domain, and lots of compile time optimization but I can see that they can be abused especially with name clashes (though gensym and hygienic macros exist), and people can't tell when an expression is evaluated. "Whitebox" macros which lookup the code of a method and use something like jquery to change code around to return a modified method are really good for debugging, and can cause some pretty nasty goto like problems if used for more than one level I can see. They can be used for static type checking that is specific to a problem, and many other stuff that is useful. I can think of many useful "whitebox" macros: add optional return values to functions, for example, the division function calculates both the reminder and the quotient, but only one is needed so it only returns one of them. So you can define several functions who have the same body but return different calculated intermediate values. Another one is implementing product (multiplying all numbers in a sequence) in terms of sum by having the macro replace + with * and 0 with 1. People tend to think in terms of specifics easier than generalities, and to understand fold (the generalization of sum and product) you need to know a specific instance of folds like sum so why not implement sum and base product on sum?

Guzman Safon
· January 23, 2020
I enjoyed the rant!

Alexander Karalis
· April 20, 2017

Actually, Parent and Child and GrandParent are not classes. That is, indeed, an incorrect use of Inheritance, but this is not a reflection on inheritance itself, but rather on how certain people misuse it. Inheritance is where subclasses are types of the superclass. So there might be a Dog class and a Golden Retriever class which inherits from Dog.

If something is-a-type-of X then you can use inheritance. Is a Child a type of Parent? No. Is a Golden Retriever a type of Dog? Yes.

Dan Gorman
· May 16, 2019
Cool post. I agree with other comments that inheritance itself isn't bad.

It's just not very useful as the cases of applying it *appropriately* and *correctly* are not so common; it's hard to do inheritance right when you factor in concepts such as Liskov Substitutability, etc.

I've often seen a language's inheritance mechanism (mis)used as a shortcut for factoring out the common code between 2 or more different classes, only to be further be referenced by that common code blob as an artificial super-type, with a lot of special checks/exceptions to compensate for the absence of a true sub-type <-> super-type relationship.

A mechanism supporting this typical pattern of common characteristics and code-sharing (such as *mixins* or *traits* as you mention) would be much more useful.

Cesar Wilson
· August 7, 2018
Rust trails are also a very nice solution to the issue of inheritance.

Reinier Post
· January 31, 2019
So what’s your stance on interfaces, e.g. in Java?

Larry Henderson
· November 20, 2019
When I think of the java class libraries for UI components I wonder where we would be without them!